A Crypto-Pipeline with Dagster

Inhaltsverzeichnis

1. Project description

Aim

The purpose of a Dagster Encryption Pipeline is to test and showcase dagsters DAG-chaining capabilities, resulting in a self-contained encryption-decryption pipeline which, 1) connects different encryption methods of a plain text prior to, 2) decrypting (and thus reversing the performed chain of encryption operations performed on an) encrypted message.

This project aims at designing a encryption-decryption pipeline via Dagsterr's website that would support users in **generating robustly encrpted messages** from a well-defined plain text input

2. Planning section

This project implements a secure and modular encryption-decryption pipeline using Dagster. It orchestrates a hybrid cryptographic workflow combining:

- **Vigenère Cipher** for classical character-level obfuscation
- **AES-256 (CBC mode)** for symmetric encryption of structured data
- **RSA-OAEP** for secure asymmetric key wrapping

The pipeline defines two primary Dagster ops:

- `encrypt_op`: Encrypts plaintext input using Vigenère, AES, and RSA layers in sequence
- `decrypt_op`: Reverses the process to retrieve the original plaintext

The `encrypt_main()` and `decrypt_main()` functions are pure, testable Python routines that can be invoked independently or orchestrated via Dagster. This makes the system both CI-friendly and scalable.

Whether you're securing short messages or experimenting with layered encryption in a data workflow context, this pipeline is a clean, educational foundation.

2.1 Structure of the encryption pipeline

Let us proceed layer by layer — from old-school ciphers to modern cryptographic standards — and unpack how each is applied algorithmically, mathematically, and in your Python implementation.

🔠 Vigenère Cipher

Mathematical Concept: The Vigenère cipher is a polyalphabetic substitution cipher that shifts each character by an amount determined by a repeating keyword. Given:

- plaintext letter (P_i)
- keyword letter (K_i)
- alphabet size (26)

The encryption function is:

$$[C_i = (P_i + K_i) \mod 26]$$

Decryption is simply:

$$[P_i = (C_i - K_i + 26) \mod 26]$$

Pythonic Implementation: In your pipeline, this is done using `ord()` for character-to-index mapping and `cycle()` from `itertools` to repeat the key:

```
for c, k in zip(plaintext.lower(), cycle(key.lower())):
    if c in alphabet:
        shifted = (ord(c) - ord('a') + ord(k) - ord('a')) % 26
        enc.append(chr(ord('a') + shifted))
```

It's efficient, readable, and makes classical encryption elegantly Pythonic.

🔐 AES-256 (CBC mode)

Mathematical Foundation: AES (Advanced Encryption Standard) is a symmetric-key block cipher based on the Rijndael algorithm. Key concepts:

- Operates on 128-bit blocks
- Uses 14 rounds for AES-256 (with 256-bit keys)
- Each round involves byte substitution (S-box), row shifting, column mixing, and key addition

CBC (Cipher Block Chaining) mode adds security by XOR'ing each block with the previous ciphertext block (with a unique IV for the first one):

$$[C_i = E_K(P_i \oplus C_{i-1})]$$

Pythonic Application: You're using `cryptography`'s `Cipher` object and PKCS7 padding to align with 128-bit blocks:

```
padder = padding.PKCS7(128).padder()
cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
```

The AES logic is clean and leverages Python's object-oriented cryptographic primitives for security and clarity.

🔑 RSA-OAEP

Mathematical Structure: RSA encryption is built on the difficulty of factoring large numbers:

$$[C = M^e \mod n \quad \text{and} \quad M = C^d \mod n]$$

OAEP (Optimal Asymmetric Encryption Padding) adds a randomized mask to the message to prevent deterministic output:

- Uses MGF1 (Mask Generation Function)
- Applies two-layered masking with a hash algorithm (e.g., SHA-256)

Pythonic Mechanics: You use OAEP with SHA-256 for padding during key wrapping (wrapping AES key with RSA):

```
public_key.encrypt(
    aes_key,
    asympadding.OAEP(
        mgf=asympadding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

This adheres to modern best practices, prevents padding oracle attacks, and ensures non-repetitive ciphertexts.

Conclusion

Each layer you've built plays a distinct role:

- Vigenère: human-readable obfuscation
- AES-256-CBC: high-speed bulk data encryption
- RSA-OAEP: secure key distribution

Together, they give your pipeline both **depth** and **defense in layers** — elegant, robust, and cryptographically sound.

2.2 Elaboration on pythonic implementation details

Layering multiple encryption techniques is a legitimate way to increase confidentiality, especially when combining reversible (symmetric/asymmetric ciphers) with non-reversible (hashing or key derivation) methods. Let's break down a pipeline concept:

🔐 Step 1: Vigenère or Caesar (Lightweight Substitution)

A first-pass classical cipher adds basic scrambling and deters casual eyes. We've already built this. An example class:

```
class VigenereCipher:
    def __init__(self, keyword: str):
        self.keyword = keyword.lower()
        self.alphabet = 'abcdefghijklmnopqrstuvwxyz'

    def shift_char(self, c, key_c, encode=True):
        if c not in self.alphabet:
            return c
        shift = self.alphabet.index(key_c)
        if not encode:
            shift = -shift
        return self.alphabet[(self.alphabet.index(c) + shift) % 26]

    def transform(self, text: str, encode=True):
        text = text.lower()
        result = []
        key_len = len(self.keyword)
        for i, c in enumerate(text):
            key_c = self.keyword[i % key_len]
            result.append(self.shift_char(c, key_c, encode))
        return ''.join(result)

    def encrypt(self, text: str) -> str:
        return self.transform(text, True)

    def decrypt(self, text: str) -> str:
        return self.transform(text, False)

if __name__ == "__main__":
    keyword = input("Enter encryption keyword: ").strip()
    cipher = VigenereCipher(keyword)

    plaintext = input("Enter text to encrypt: ").strip()
    encrypted = cipher.encrypt(plaintext)
    print(f"Encrypted text: {encrypted}")

    decrypted = cipher.decrypt(encrypted)
    print(f"Decrypted back: {decrypted}")
```

🔁 Step 2: Symmetric Encryption (AES or ChaCha20)

Use a **modern symmetric cipher** like:

- **AES** (Advanced Encryption Standard) — widely adopted, secure, hardware-accelerated
- **ChaCha20** — faster in software and more resistant to timing attacks

🛠️ We would need a secure key and an Initialization Vector (IV) — both must be handled carefully.

🔑 Step 3: Asymmetric Encryption (RSA or ECC)

Wrap the **AES/ChaCha key using RSA** (public-key crypto). That way:

- Only the intended recipient (with the private key) can unwrap the symmetric key.
- This hybrid approach is how HTTPS and encrypted messaging apps work.

🔁 Step 4: Hashing + Signature (Optional but Powerful)

- **Hash (SHA-256)** the original text for **integrity checking**
- Sign the hash using a private key (e.g., **RSA signature** or **ECDSA**) to verify authenticity

🔁 Bonus: Polymorphic Layer

A **polymorphic cipher** randomly alters its encoding strategy per encryption session — changing keys, routes, or even algorithm subsets. It's excellent for:

- Detecting tampering or replay
- Obfuscation-based defense (especially in malware counter-analysis)

📦 Example Pipeline Flow

```
[Plaintext]
   ↓ Vigenère (keyword obfuscation)
[Step 1 Output]
   ↓ AES (symmetric block encryption)
[Step 2 Output]
   ↓ RSA public key encryption (protect AES key)
[Encrypted + EncryptedKey]
   ↓ Hash + Sign (SHA-256 + RSA signature)
[Final Output Package]
```

3. Pythonic Crypto pipeline implementation

Here’s how we can approach this project step by step:

3.1 Packages to be installed

```
!pip install cryptography
!pip install pycryptodome
!pip install typer rich
!pip install dagster dagster-webserver

# Optional: for Jupyter support
!pip install dagster[jupyter]

# Optional: for filesystem and database IO management
!pip install dagster-io

# Optional: include example integrations
!pip install dagster pandas requests
```

3.2 Generate private and public keys

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
import os

# Ensure the rsa/ directory exists
os.makedirs("rsa", exist_ok=True)

# Generate private key
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)

# Save private key
with open("root/rsa/private.pem", "wb") as f:
    f.write(
        private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption()
        )
    )

# Save public key
public_key = private_key.public_key()
with open("root/rsa/public.pem", "wb") as f:
    f.write(
        public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        )
    )
```

Remark: These keys have to be generated simultaneously within the same folder in order for the pipeline to operate flawlessly.

3.3 Encryption pipeline

The encryption pipeline code may look as follows:

```
def vigenere_encrypt(plaintext: str, key: str) -> str:
    from itertools import cycle
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    enc = []
    for c, k in zip(plaintext.lower(), cycle(key.lower())):
        if c in alphabet:
            shifted = (ord(c) - ord('a') + ord(k) - ord('a')) % 26
            enc.append(chr(ord('a') + shifted))
        else:
            enc.append(c)
    return ''.join(enc)

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import default_backend
import os

def aes_encrypt(data: bytes, key: bytes) -> tuple[bytes, bytes, bytes]:
    iv = os.urandom(16)
    padder = padding.PKCS7(128).padder()
    padded_data = padder.update(data) + padder.finalize()
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    encryptor = cipher.encryptor()
    ct = encryptor.update(padded_data) + encryptor.finalize()
    return ct, iv, key

from cryptography.hazmat.primitives.asymmetric import rsa, padding as asympadding
from cryptography.hazmat.primitives import serialization, hashes

def rsa_encrypt_key(aes_key: bytes, public_key) -> bytes:
    return public_key.encrypt(
        aes_key,
        asympadding.OAEP(
            mgf=asympadding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
```

```python
from cryptography.hazmat.primitives.asymmetric import padding as asypad
from cryptography.hazmat.primitives import hashes

def sign_data(private_key, message: bytes) -> bytes:
    return private_key.sign(
        message,
        asypad.PSS(
            mgf=asypad.MGF1(hashes.SHA256()),
            salt_length=asypad.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

import argparse

def main():
    parser = argparse.ArgumentParser(description="Secure Encryption Pipeline")
    parser.add_argument("text", help="Text to encrypt")
    parser.add_argument("--vkey", required=True, help="Vigenère keyword")
    args = parser.parse_args()

    # Step 1: Vigenère
    v_encrypted = vigenere_encrypt(args.text, args.vkey)

    # Step 2: AES
    aes_key = os.urandom(32)
    aes_ct, iv, key = aes_encrypt(v_encrypted.encode(), aes_key)

    # Step 3: RSA
    # Load/generate RSA keys here
    # rsa_ct = rsa_encrypt_key(aes_key, public_key)

    print("Encrypted Text:", aes_ct.hex())

if __name__ == "__main__":
    main()
```

Example usage (bash)

Here's an example of how one could use the encryption pipeline from a terminal, along with a demonstration of what happens at each step using mock data:

✅ Step 1: Save the Script

Save your pipeline code into a Python file, for example:

```
secure_pipeline.py
```

✅ Step 2: Run It From CLI

```
python secure_pipeline.py "This is top secret text." --vkey lemon
```

This triggers:

1. **Vigenère Encryption** of the plaintext using the keyword `lemon`
   Output (example): `aohu wp hsd fvuvy vica.`
2. **AES Encryption** of the result (binary output encoded as hex)
   Output (example): `afcb0192b8eaeffa1e3435a4da02c976f65411a7f71af5a56ce10833e6785021`
3. **RSA Encryption** (optional — in your code it's currently commented out)
4. **Digital Signing** of the message (optional — defined but not yet called)

🔐 Example Output (simplified)

```
Encrypted Text: afcb0192b8eaeffa1e3435a4da02c976f65411a7f71af5a56ce10833e6785021
```

💡 Notes for Extension

- To make the pipeline fully functional, we should:
  - Load or generate a public/private RSA key pair
  - Implement `rsa_encrypt_key(aes_key, public_key)` and print/store the result
  - Store or transmit the IV and the RSA-encrypted AES key securely
  - Optionally sign the AES ciphertext with `sign_data(...)`

3.4 Decryption pipeline

The decryption pipeline code may look as follows:

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding, serialization, hashes
from cryptography.hazmat.primitives.asymmetric import padding as asympadding
from cryptography.hazmat.backends import default_backend
from itertools import cycle
import argparse

# Step 1: Vigenère Decryption
def vigenere_decrypt(ciphertext: str, key: str) -> str:
    alphabet = 'abcdefghijklmnopqrstuvwxyz'
    plaintext = []
    for c, k in zip(ciphertext.lower(), cycle(key.lower())):
        if c in alphabet:
            shift = (ord(c) - ord(k)) % 26
            plaintext.append(chr(ord('a') + shift))
        else:
            plaintext.append(c)
    return ''.join(plaintext)

# Step 2: AES Decryption
def aes_decrypt(ciphertext: bytes, key: bytes, iv: bytes) -> bytes:
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    decryptor = cipher.decryptor()
    padded_data = decryptor.update(ciphertext) + decryptor.finalize()
    unpadder = padding.PKCS7(128).unpadder()
    return unpadder.update(padded_data) + unpadder.finalize()

# Step 3: RSA Key Decryption
def rsa_decrypt_key(encrypted_key: bytes, private_key_path: str) -> bytes:
    with open(private_key_path, "rb") as key_file:
        private_key = serialization.load_pem_private_key(
            key_file.read(),
            password=None,
            backend=default_backend()
        )
    return private_key.decrypt(
        encrypted_key,
        asympadding.OAEP(
            mgf=asympadding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
```

```python
# Main CLI logic
def main():
    parser = argparse.ArgumentParser(description="Secure Decryption Pipeline")
    parser.add_argument("ciphertext", help="AES ciphertext as hex string")
    parser.add_argument("--iv", required=True, help="AES initialization vector (hex)")
    parser.add_argument("--rsa_key", required=True, help="Path to private RSA PEM key")
    parser.add_argument("--enc_key", required=True, help="RSA-encrypted AES key (hex)")
    parser.add_argument("--vkey", required=True, help="Vigenère keyword used at encryption")
    args = parser.parse_args()

    # Step 1: Decrypt AES key with RSA
    encrypted_key = bytes.fromhex(args.enc_key)
    aes_key = rsa_decrypt_key(encrypted_key, args.rsa_key)

    # Step 2: Decrypt AES-encrypted Vigenère text
    ciphertext = bytes.fromhex(args.ciphertext)
    iv = bytes.fromhex(args.iv)
    vigenere_encrypted = aes_decrypt(ciphertext, aes_key, iv).decode()

    # Step 3: Decrypt Vigenère to get plaintext
    original_text = vigenere_decrypt(vigenere_encrypted, args.vkey)

    print("\n✅ Decrypted Plaintext:", original_text)

if __name__ == "__main__":
    main()
```

Example usage (bash)

```bash
python decrypt_pipeline.py \
  "62c99f..." \
  --iv "3f3ec2..." \
  --rsa_key private_key.pem \
  --enc_key "45ff19..." \
  --vkey lemon
```

4. Dagster Pipeline structure

We are ready to bring in Dagster to orchestrate your encryption and decryption flows, here's how one can wire it all together. We'll treat `secure_pipeline.py` and `decrypt_pipeline.py` as modular components and then use Dagster to manage the flow, parameters, and runtime configuration.

Dagster isn't just a tool for orchestrating data pipelines — it's a framework that encourages a thoughtful, *composable*, and *observable* approach to building reliable workflows.

🧠 Dagster's Core Philosophy

At its heart, Dagster is built around three transformative ideas:

1. **Software-defined assets (SDAs)**:
    - Instead of thinking in terms of abstract steps or jobs, Dagster encourages you to define the *data itself* as first-class citizens.
    - You define assets and how they're built — this shifts the focus from *how to run tasks* to *what data artifacts are produced and how they depend on each other*.
2. **Declarative over imperative thinking**:
    - Traditional pipeline tools often define "what to do step by step." Dagster, on the other hand, lets you declare the relationships and let the system figure out optimal execution.
    - Dependencies (inputs and outputs) are tracked explicitly.
3. **Observability, testing, and ergonomics from day one**:
    - Dagster treats reliability, logging, introspection, and local testing as core concerns — not afterthoughts.
    - You can dry-run, inspect intermediate values, retry failed steps, visualize asset lineage, and more — all with powerful tooling like Dagit.

⚙️ How Dagster Works

Your pipeline is made of:

- **@asset or @op**: The individual units of computation. Think of an `@op` as a function that's trackable, testable, and observable. `@asset` declares higher-level data dependencies.
- **Graphs / Jobs**: You group `ops` or `assets` into logical workflows using `@job`.
- **Dagit**: The browser-based UI for running, inspecting, and debugging pipelines. It offers a beautiful DAG view of your workflows and interactive logs.
- **IO Managers / Resources**: Pluggable hooks to handle where your data goes (memory, filesystem, database), and how you connect to external services (like S3, Spark, PostgreSQL, etc.).

💥 What Makes Dagster Different

- **Dev-first & CI-friendly**: Local workflows mirror production; it plays nicely with Git.
- **Context-aware execution**: Each op receives a structured context (e.g. logs, config, resources) rather than relying on global state.
- **Robust type system**: You can define schemas, validation, and structured inputs/outputs — it's like the joy of Python typing, but for pipeline data.
- **Incremental builds**: It supports re-materializing only the assets that are out-of-date — like `make` for data pipelines.

In short, Dagster reimagines what a data orchestration tool should be: *introspective, testable,* and *built for humans who like clean code and confidence in deployment.*

4.1 Project Structure

```
project_root/
|
├── pipelines/
|   ├── secure_pipeline.py        # contains encrypt logic
|   ├── decrypt_pipeline.py       # contains decrypt logic
|
├── dagster_pipelines/
|   ├── __init__.py
|   ├── jobs.py                   # Dagster job definitions
|   └── ops.py                    # Dagster steps
|
├── workspace.yaml
└── dagster.yaml
```

4.2 Define Dagster Ops (ops.py)

```python
from dagster import op
from pipelines import secure_pipeline, decrypt_pipeline

@op
def encrypt_op(context, text: str, vkey: str):
    return secure_pipeline.encrypt_main(text, vkey)  # create `encrypt_main()` in secure_pipeline.py

@op
def decrypt_op(context, encrypted_data: dict, vkey: str, rsa_key_path: str):
    return decrypt_pipeline.decrypt_main(
        ciphertext=encrypted_data['ciphertext'],
        iv=encrypted_data['iv'],
        enc_key=encrypted_data['enc_key'],
        priv_key_path=rsa_key_path,
        vkey=vkey
    )
```

4.3 Define Dagster Jobs (jobs.py)

```python
from dagster import job
from .ops import encrypt_op, decrypt_op

@job
def encryption_decryption_job():
    decrypted = decrypt_op(
        encrypted_data=encrypt_op(),
        vkey="lemon",
        rsa_key_path="rsa_keys/private_key.pem"
    )
```

We can also parameterize the job with `@configurable` or use Dagster's config schema if you want to expose CLI/graph inputs.

4.4 Entry Point (CLI or UI)

From the terminal:

```
dagster dev   # launches Dagster UI
```

Or from a Python CLI:

```
from dagster_pipelines.jobs import encryption_decryption_job
encryption_decryption_job.execute_in_process()
```

4.5 Structure of pythonic crypto-files

Each should expose a callable like this:

```
# secure_pipeline.py
def encrypt_main(text: str, vkey: str) -> dict:
    # return {
    #   "ciphertext": <hex>,
    #   "iv": <hex>,
    #   "enc_key": <hex>
    # }
```

```
# decrypt_pipeline.py
def decrypt_main(ciphertext, iv, enc_key, priv_key_path, vkey) -> str:
    # return plaintext
```

This turns our CLI tools into reusable data pipeline steps. Add logging, tests, or run schedules and we on our way to production-grade cryptography orchestration.

5. Pipeline execution via dagster

To get up and running with **Dagster**, we will want to install its core package along with any extras you plan to use (like AWS, dbt, Airbyte, etc). But for a basic local development setup with the web UI, here's what we need:

```
!pip install dagster dagster-webserver
```

This gives us:

- `dagster`: core library for defining jobs, ops, resources, etc.
- `dagster-webserver`: the lightweight UI you can launch with `dagster dev`

If we are using extras like Jupyter, SQLAlchemy, or cloud integrations, you can layer in specific extras:

```
# Optional: for Jupyter support
!pip install dagster[jupyter]

# Optional: for filesystem and database IO management
!pip install dagster-io

# Optional: include example integrations
!pip install dagster pandas requests
```

Once installed, we launch our dev server like this in a terminal:

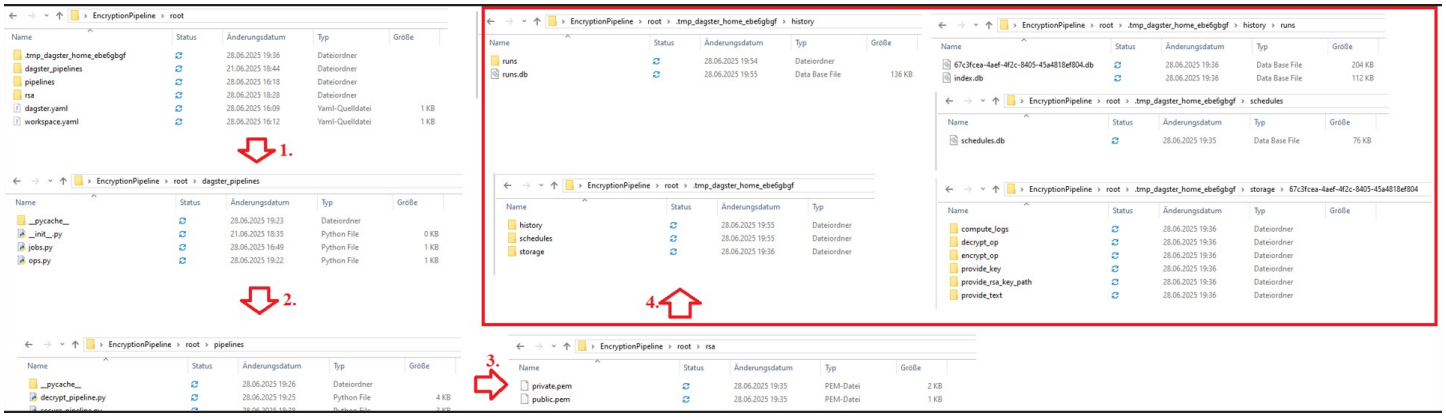
```
dagster dev
```

And our UI will be live at http://localhost:3000 🎲

5.1 Package installation

```
!pip install cryptography
!pip install pycryptodome
!pip install typer rich
!pip install dagster dagster-webserver

# Optional: for Jupyter support
!pip install dagster[jupyter]

# Optional: for filesystem and database IO management
!pip install dagster-io

# Optional: include example integrations
!pip install dagster pandas requests
```

5.2 Generate appropriate folder structure



5.3 Generate public and private keys

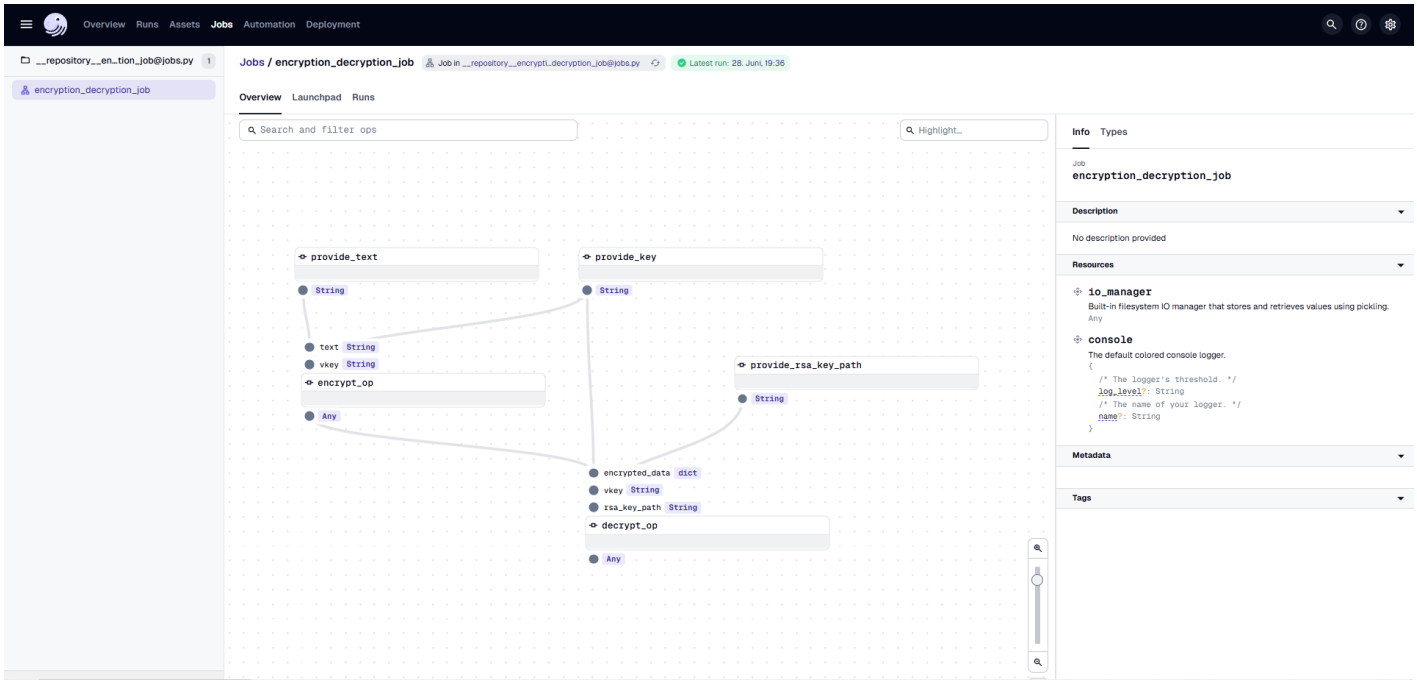
```
`from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
import os

# Ensure the rsa/ directory exists
os.makedirs("rsa", exist_ok=True)

# Generate private key
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)

# Save private key
with open("root/rsa/private.pem", "wb") as f:
    f.write(
        private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption()
        )
    )

# Save public key
public_key = private_key.public_key()
with open("root/rsa/public.pem", "wb") as f:
    f.write(
        public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        )
    )
```

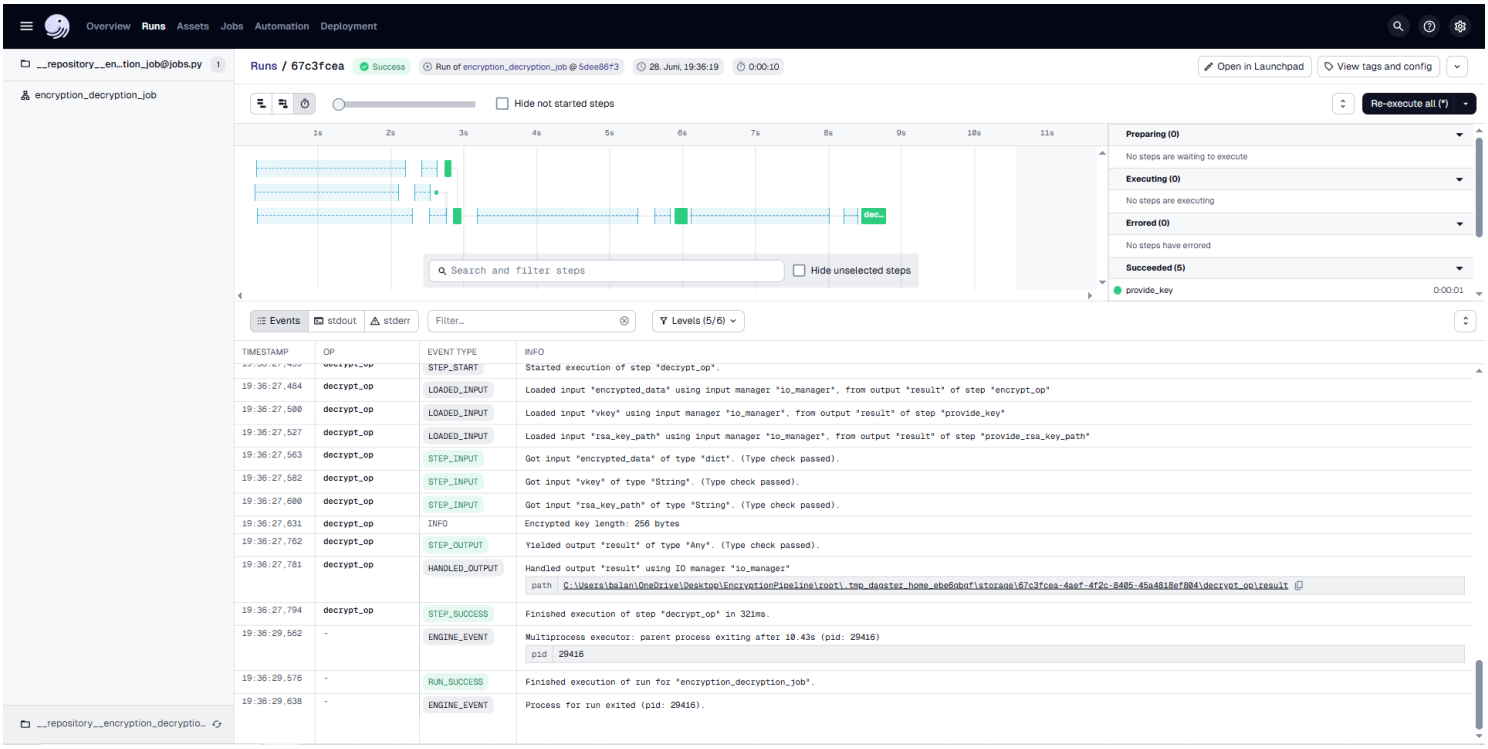
5.4 Execute the Dagster UI

```
import os
print(os.getcwd()) # Verify the existence of our current working directory

# Chnage to the pwd location
os.chdir("C:\\Users\\balan\\OneDrive\\Desktop\\EncryptionPipeline\\root")

# Execute the dagster-pipeline
!dagster dev -w workspace.yaml
```

5.5 Open the Dagster UI and run jobs

Our UI will be live at http://localhost:3000



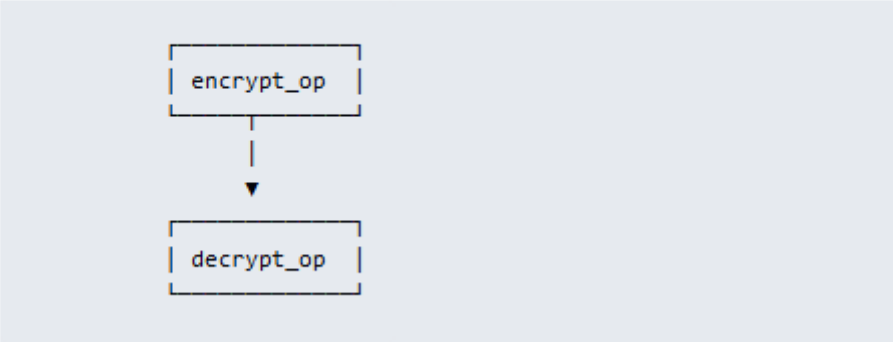

Overview Runs Assets Jobs Automation Deployment
__repository__en..tion_job@jobs.py
encryption_decryption_job
Runs / 67c3fcea
Success
Run of encryption_decryption_job @ 5dee86f3
28. Juni, 19:36:19
0:00:10
Open in Launchpad
View tags and config
Hide not started steps
Re-execute all (*)
Search and filter steps
Hide unselected steps
Preparing (0)
No steps are waiting to execute
Executing (0)
No steps are executing
Errored (0)
No steps have errored
Succeeded (5)
provide_key 0:00:01
Events
stdout
stderr
Filter...
Levels (5/6)
TIMESTAMP OP EVENT TYPE INFO
STEP_START Started execution of step "decrypt_op".
19:36:27,484 decrypt_op LOADED_INPUT Loaded input "encrypted_data" using input manager "io_manager", from output "result" of step "encrypt_op"
19:36:27,500 decrypt_op LOADED_INPUT Loaded input "vkey" using input manager "io_manager", from output "result" of step "provide_key"
19:36:27,527 decrypt_op LOADED_INPUT Loaded input "rsa_key_path" using input manager "io_manager", from output "result" of step "provide_rsa_key_path"
19:36:27,563 decrypt_op STEP_INPUT Got input "encrypted_data" of type "dict". (Type check passed).
19:36:27,582 decrypt_op STEP_INPUT Got input "vkey" of type "String". (Type check passed).
19:36:27,600 decrypt_op STEP_INPUT Got input "rsa_key_path" of type "String". (Type check passed).
19:36:27,631 decrypt_op INFO Encrypted key length: 256 bytes
19:36:27,762 decrypt_op STEP_OUTPUT Yielded output "result" of type "Any". (Type check passed).
19:36:27,781 decrypt_op HANDLED_OUTPUT Handled output "result" using IO manager "io_manager"
path C:\Users\balan\OneDrive\Desktop\EncryptionPipeline\root\.tmp_dagster_home_ebe6qbof\storage\67c3fcea-4aef-4f2c-8405-45a4818ef804\decrypt_op\result
19:36:27,794 decrypt_op STEP_SUCCESS Finished execution of step "decrypt_op" in 321ms.
19:36:29,562 - ENGINE_EVENT Multiprocess executor: parent process exiting after 10.43s (pid: 29416)
pid 29416
19:36:29,576 - RUN_SUCCESS Finished execution of run for "encryption_decryption_job".
19:36:29,638 - ENGINE_EVENT Process for run exited (pid: 29416).
__repository__encryptio...

6. Resume

Let us visualize how our encryption-decryption pipeline fits into Dagster's elegant DAG (Directed Acyclic Graph) philosophy.

6.1 The DAG view of the Crypto pipeline

Our Dagster job can be visualized as a simple, two-op DAG:



6.2 Op Breakdown

- `encrypt_op`
  - Input: plaintext string and Vigenère key
  - Flow:
    a. Applies the Vigenère cipher
    b. Generates a random AES-256 key
    c. Encrypts the Vigenère text with AES in CBC mode
    d. Encrypts the AES key using an RSA public key (OAEP)
  - Output: dictionary containing:
    - ciphertext
    - initialization vector (IV)
    - RSA-encrypted AES key
- `decrypt_op`
  - Input: the dictionary above, private RSA key path, and Vigenère key
  - Flow:
    a. Decrypts the AES key using RSA private key
    b. Uses the AES key + IV to decrypt the ciphertext into Vigenère text
    c. Applies Vigenère decryption to recover original plaintext
  - Output: restored message (string)

6.3 How Dagster improves procedures

- **Visual traceability:** In Dagit, we see this pipeline clearly mapped, so you know exactly what data flows where.
- **Logs and retries:** If, say, the decryption step fails (e.g., due to mismatched keys), only `decrypt_op` needs to be retried — `encrypt_op`'s output can be reused.
- **Dagster context:** Each op can log intermediate results using `context.log`, making debugging crypto internals much easier.
- **Unit testable:** Because each op wraps around `encrypt_main()` and `decrypt_main()`, you can test them in isolation or as a job.

6.4 Summary

Overview

This project implements a modular, multi-layered encryption and decryption pipeline using **Python** and **Dagster**. It combines three cryptographic layers — a classical Vigenère cipher, industry-standard AES-256 in CBC mode, and RSA-OAEP key encapsulation — into a single orchestrated workflow. Dagster's type-safe, observable framework powers clean data lineage, easy debugging, and end-to-end reproducibility.

Pipeline Components

1. **Vigenère Cipher (Python-based Classic Cipher)**
    - Applies polyalphabetic substitution at the character level
    - Adds lightweight obfuscation prior to cryptographic encryption
    - Fully reversible for decryption step
2. **AES-256 (CBC Mode)**
    - Uses a 256-bit randomly generated key for symmetric encryption
    - Ensures confidentiality with PKCS7 padding and secure IV generation
    - Implemented using the `cryptography` library for robustness
3. **RSA-OAEP**
    - Encrypts the AES key using an RSA public key
    - Decrypts it using the matching private key
    - Guarantees secure key exchange and resistance to chosen ciphertext attacks

Dagster Integration

- **@op Decorators:**
  - `encrypt_op`: Wraps the encryption routine
  - `decrypt_op`: Wraps the decryption logic with error-safe RSA and AES reversal
- **@job Composition:**
  - `encryption_decryption_job`: A sequential DAG that securely encrypts data and then verifies its integrity by decrypting it
- **Dagit Support:**
  - Real-time DAG visualization
  - Execution logs and intermediate values
  - Easy testing and local development

Use Cases

- Educational tool for layered cryptography
- Modular base for secure communication pipelines
- Blueprint for adding encryption to ETL workflows or asset versioning

7. References

1. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).

2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", Mcgraw-Hill, 6th Edition (2010).

3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).

4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.

5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).

6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).

7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).

8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).

9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).

10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).

11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).

12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).

13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).

14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).

15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).

16. Volker Ziemann: "Physics and Finance", Springer (2021).

17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).

18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).

19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).

20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).

21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).

22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).

23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).

24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).

25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).

26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).

27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).

28. David Foster: "Generative Deep Learning", O'Reilly(2023).

29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).

30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).

31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).

32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).

33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).